COSC 3430 · ARTIFICIAL INTELLIGENCE

End-to-End Machine Learning Project

Part 3: create a test set, before you look any closer

ROADMAP

Ten steps, over several code-along lectures

1 Look at the big picture
2 Get the data
3 Explore and visualize the data
4 Create the test data
5 Prepare the data for learning
6 Select a model and train it
7 Fine-tune the model
8 Present your solution
9 Launch the production system
10 Monitor and maintain it

Today: step 4 - then back to step 3, on the training set only.

See also: Géron's ML project checklist (github.com/ageron/handson-mlp)

QUESTION

Why set the test set aside before exploring the data?

ANSWER

Biased data means a fake performance measure

We evaluate the model on data it has never seen to avoid **bias**: a test set that doesn't represent the population gives a performance measure that's fake - the model looks as if it generalizes well, but it doesn't.

No data snooping

The test set must be set aside **before** any exploration. Looking at it first biases your intuitions and your model choices.

→ SWITCH TO EDITOR · CODE-ALONG 1

Split the data at random

1. Import `train_test_split` from scikit-learn
2. Hold out 20% of the districts as the test set
3. Fix `random_state=42` so the split is reproducible

```
train_set, test_set = train_test_split(housing_full, test_size=0.2, random_state=42)
```

QUESTION 1

Housing prices depend heavily on a district's median income. If you pick 20% of districts purely at random, is there any guarantee this test set's income mix looks like the full dataset's? What if you only had 200 districts instead of 20,640?

QUESTION 2

You add 500 new houses next month and rerun this exact code. Does the test set still contain the same houses as before — and does that matter?

ANSWER

Two problems with random sampling

ANSWER 1 · SAMPLING BIAS

Why a random 20% isn't safe

On a small dataset, a random 20% may not represent the full population — a handful of unlucky rows can skew the test set's income or location mix

ANSWER 2 · THE RESHUFFLING TRAP

No fixed split, no stable test set

Without a fixed random_state, each rerun draws a different 20% - over many reruns the model indirectly sees the whole dataset. Pinning random_state=42 helps, but a refreshed dataset still shifts which rows land in that 20%, making the split stale

Robust fix: hashing

Hash a stable id (row index, district id) per row: the same instances stay in the same set, regardless of order or dataset size

A hash is a unique key per row: deterministic, stable across updates

QUESTION

Even with a stable hash, why is random sampling still risky for this dataset?

ANSWER

Income is correlated with price

Both the test and training sets should represent the full population of CA districts. Domain experts say **median_income** is highly correlated with house value.

The risk

If the test set randomly undersamples high- or low-income districts, the error estimates become systematically off - even though the sampling itself was fair.

Random gets the **size** of the test set right, not necessarily the **mix** - and income is exactly the mix that matters here.

Fix: sample within income strata, not across the whole dataset

→ SWITCH TO EDITOR · CODE-ALONG 2

Cut median_income into categories

1. Use `pd.cut()` to bucket the continuous income values
2. Five bins: `0, 1.5, 3, 4.5, 6, ∞`
3. Label the bins 1–5 as `income_cat`

```
income_cat = pd.cut(median_income, bins=[0,1.5,3,4.5,6,inf], labels=[1,2,3,4,5])
```

QUESTION

Why can't we stratify on the raw income values?

ANSWER

Too many, too small strata

- Continuous values like 3.2741, 5.1823, 2.9004 - how many districts share these?
- **None.** Every district would be its own stratum
- Too small to meaningfully sample from

income_cat	share of districts
1	4.0%
2	31.9%
3	35.1%
4	17.6%
5	11.4%

Category 3 dominates; 1 and 5 are rare but not empty - well-populated enough to stratify on

→ SWITCH TO EDITOR · PAIR ACTIVITY

Split within each category, and verify

Work with a partner. One of you drives the keyboard, the other reads the steps aloud.

1. Pass `stratify=income_cat` to `train_test_split`
2. Compare category proportions, full vs. test set
3. They should match closely - that's the point

```
strat_train_set, strat_test_set = train_test_split(…, stratify=income_cat, random_state=42)
```

RESULT

Proportions match, in train and test

income_cat	full dataset	test set
3	0.350581	0.350533
2	0.318847	0.318798
4	0.176308	0.176357
5	0.114438	0.114341
1	0.039826	0.039971

- **Stratified sampling** preserves the income distribution in both sets - plain random sampling doesn't guarantee this, especially in the rare tails (1 and 5)
- This proportion comparison is what "representative" means, empirically

→ SWITCH TO EDITOR · CODE-ALONG 4

Drop the helper column

1. `income_cat` was only scaffolding for the split
2. Drop it from both `strat_train_set` and `strat_test_set`
3. The test set is now sealed - don't touch it again

```
for df in (strat_train_set, strat_test_set): df.drop("income_cat", axis=1, inplace=True)
```

WRAP-UP

The test set is sealed - here's why we trust it

- Set aside before any exploration: no data snooping
- Plain random sampling risks bias, and reshuffles on every rerun
- Stratified sampling on median_income keeps both sets representative

Next: explore and visualize - on strat_train_set only.